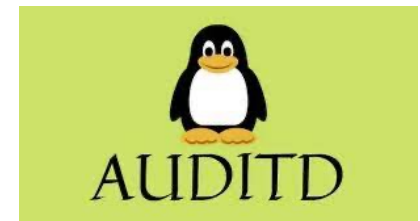
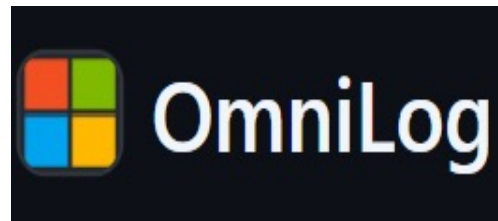
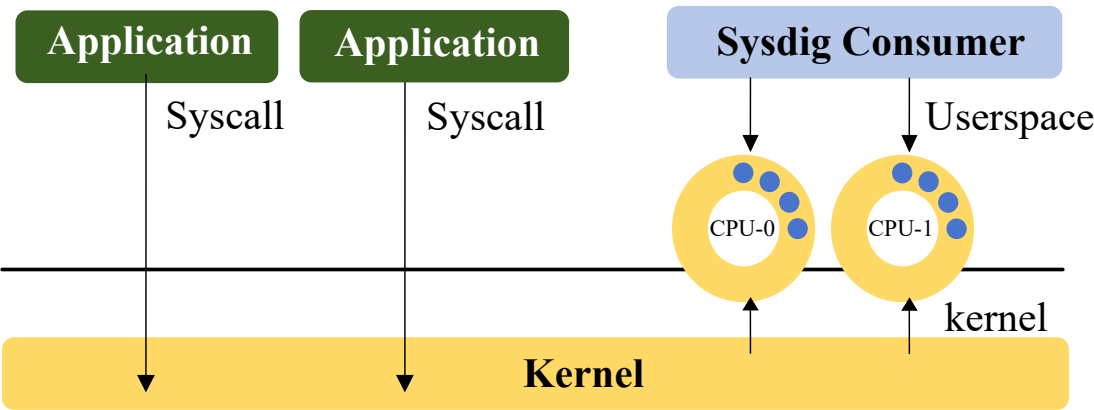


Auditing Framework

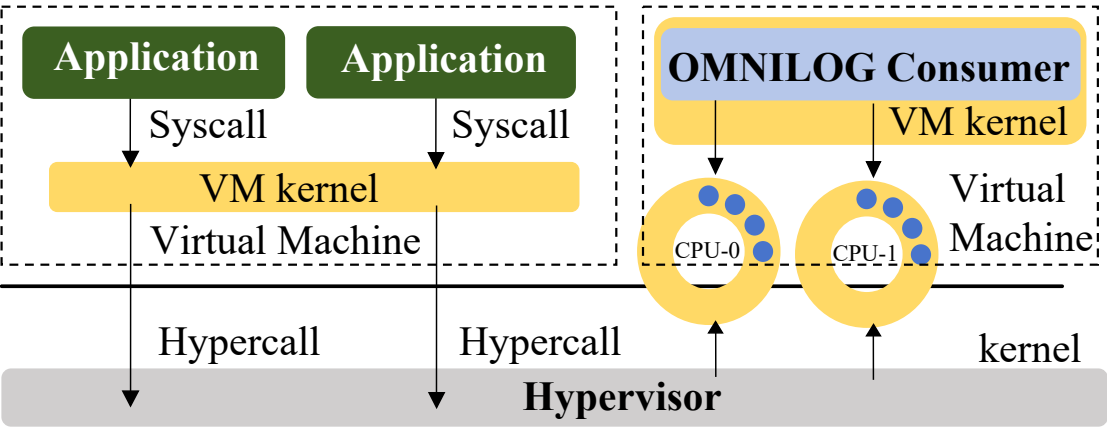
- **Auditing framework** is critical operating system security infrastructure for intrusion detection, provenance analysis, compliance verification, and attack reconstruction.
- SoTA: Sysdig, OMNILOG, Ellipsis, Linux Audit.



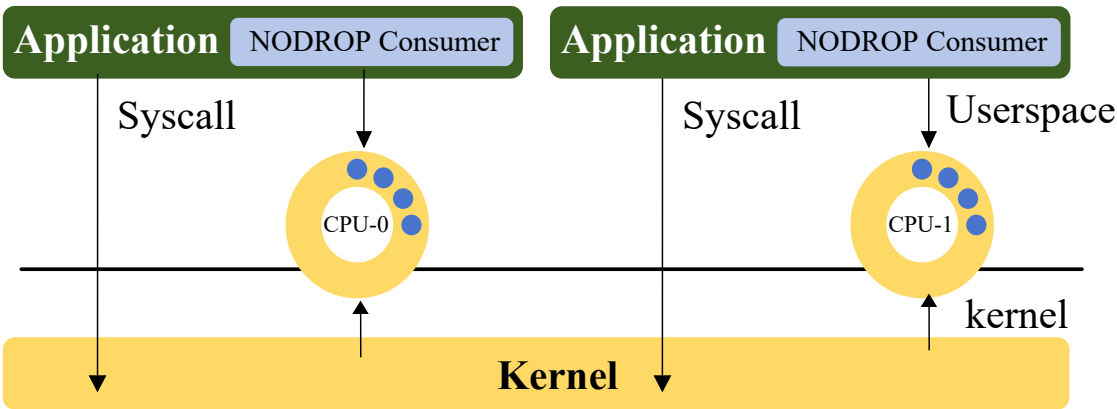
Popular Auditing Frameworks



(a) Sysdig and Linux Auditd

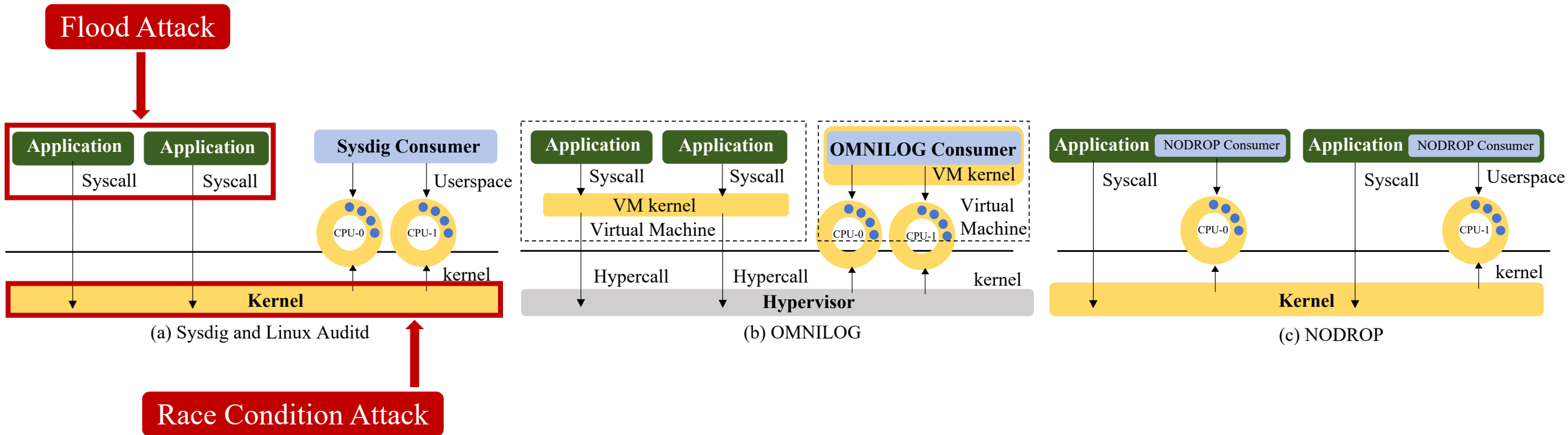


(b) OMNILOG



(c) NODROP

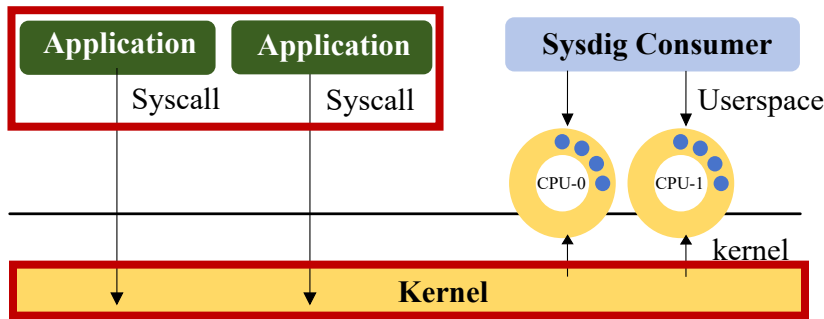
Security Threats on Auditing Frameworks



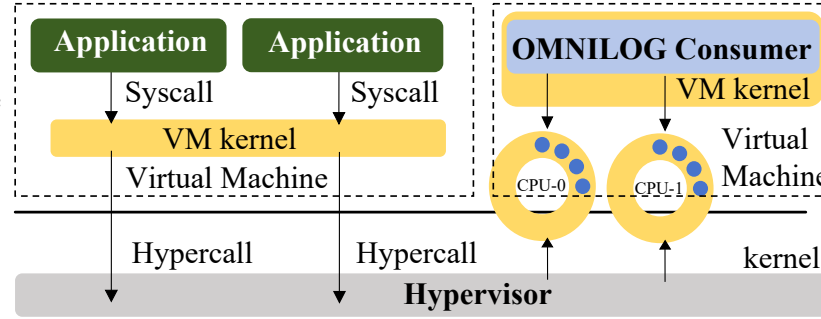
- **Race Condition Attack:** An attack can delete audit events generated by malicious processes from the kernel buffers before they have been submitted to the consumer.
- **Flood Attack:** An attacker can quickly generate a large number of invalid system logs, crowd the log buffer, and cause the subsequent attack log loss. (Also taken as PDoS attack)

Existing Auditing Frameworks

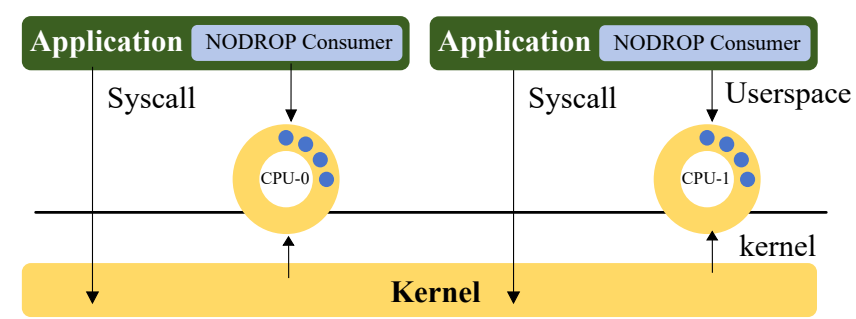
Weakness Overview



(a) Sysdig and Linux Auditd



(b) OMNILOG



(c) NODROP

Sysdig, Linux Auditd, Ellipsis:

- Unbounded event residence time
- Event dropping
- Priority inversion

OMNILOG:

Substantial execution overhead

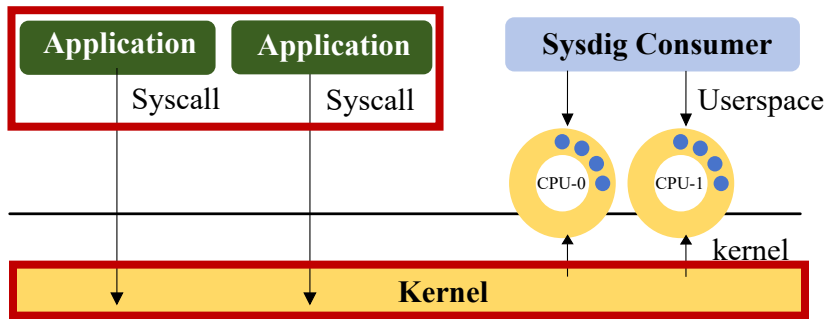
- syscall overhead,
- virtualization overhead.

NODROP:

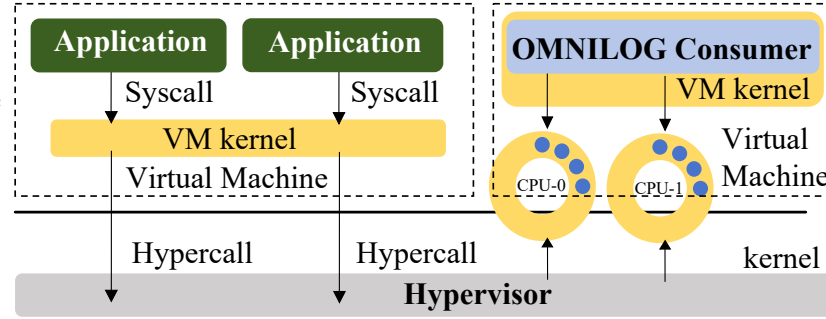
- Large consumer interference
- Burst event processing
- Long time residence time.

Existing Auditing Frameworks

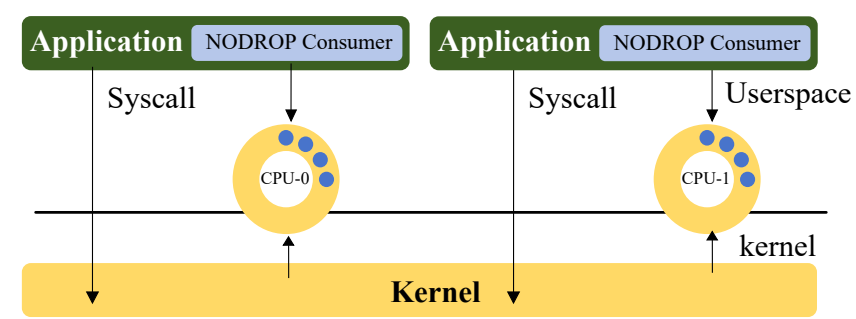
Weakness



(a) Sysdig and Linux Auditd



(b) OMNILOG



(c) NODROP

It's essential for creating a safe, predictable and efficient Auditing Framework to guarantee Real-Time performance.

Research Goals

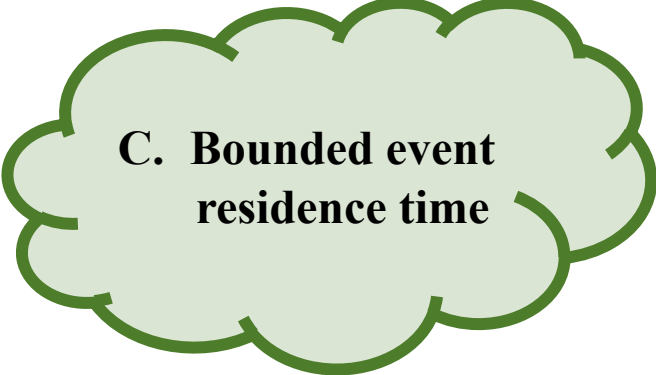
An auditing framework for real-time systems needs to match these following goals.



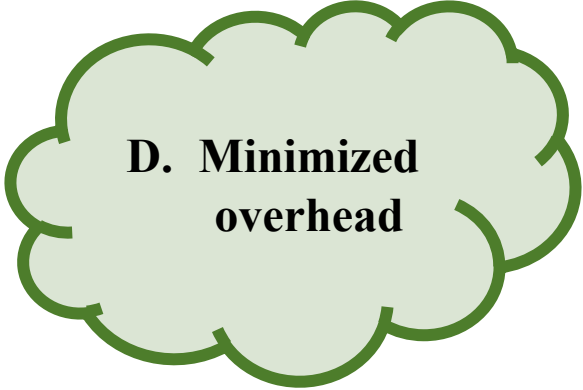
A. Event integrity



**B. Guaranteed
schedulability**

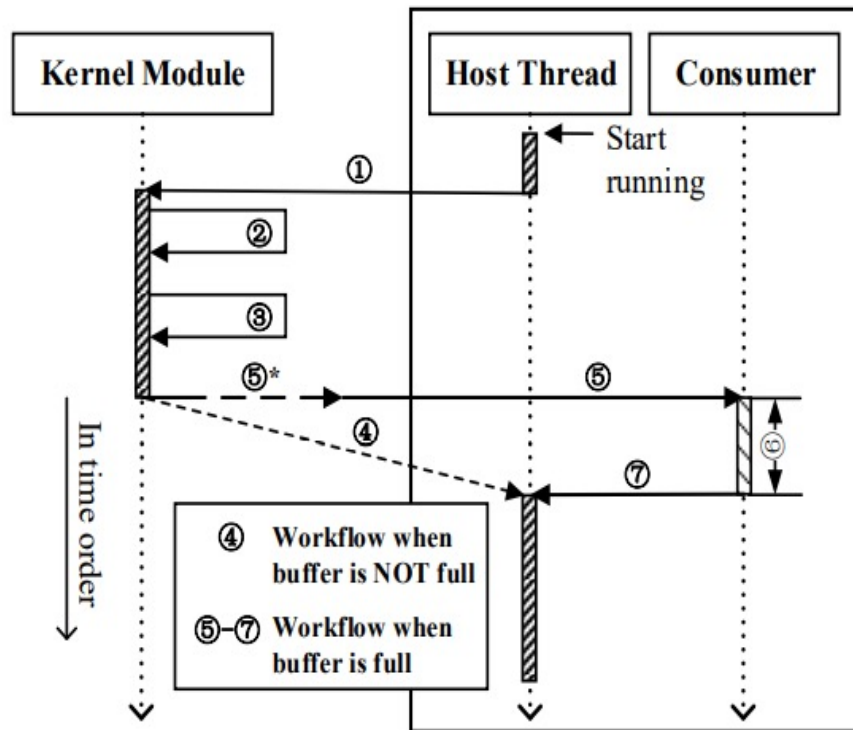


**C. Bounded event
residence time**



**D. Minimized
overhead**

Real-Time Auditing Implementation



NODROP Workflow

S1: The kernel captures a system call (①) and executes it (②). System call is recored in a dedicated in-kernel logging buffer (③).

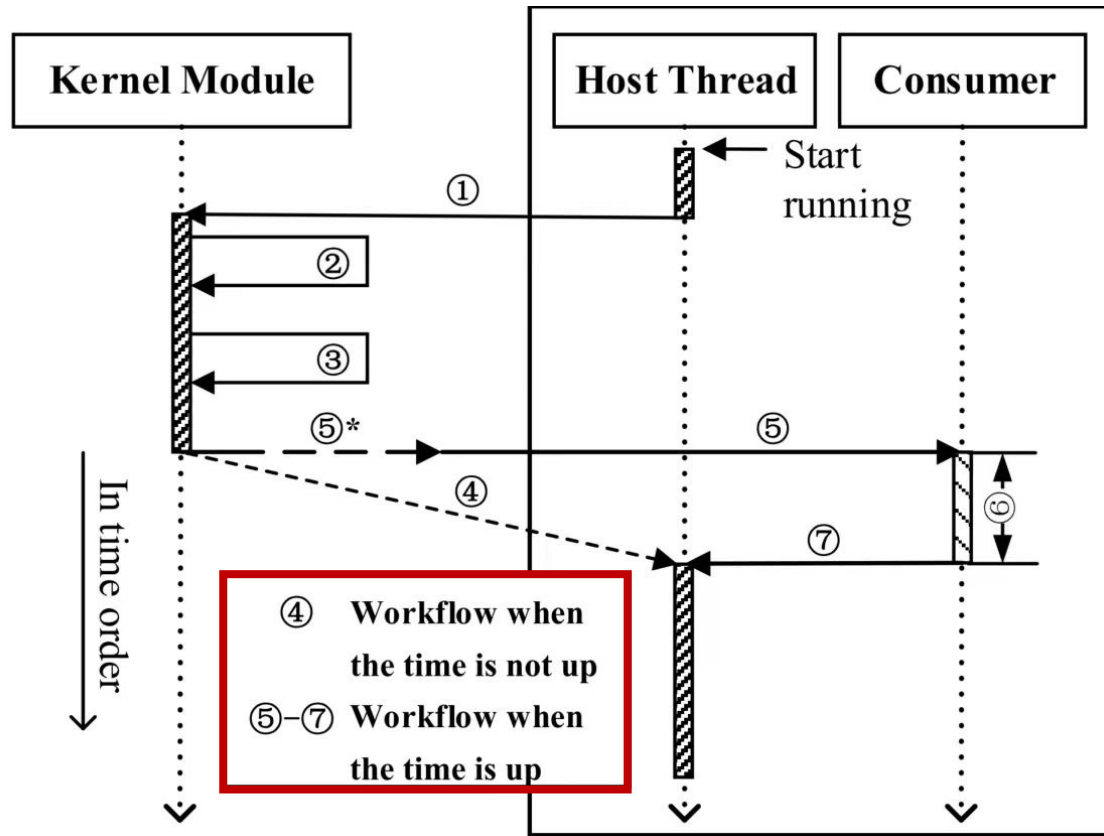
S2: Buffer is not full: returns control to the host thread (④). Buffer is full or the thread exits: control is passed to the consumer (⑤).

S3: Consumer is instrumented into the running thread(⑤*).

S4: Consumer process the transferred provenance data (⑥).

S5: Control is returned to the original thread (⑦).

Real-Time Auditing Implementation



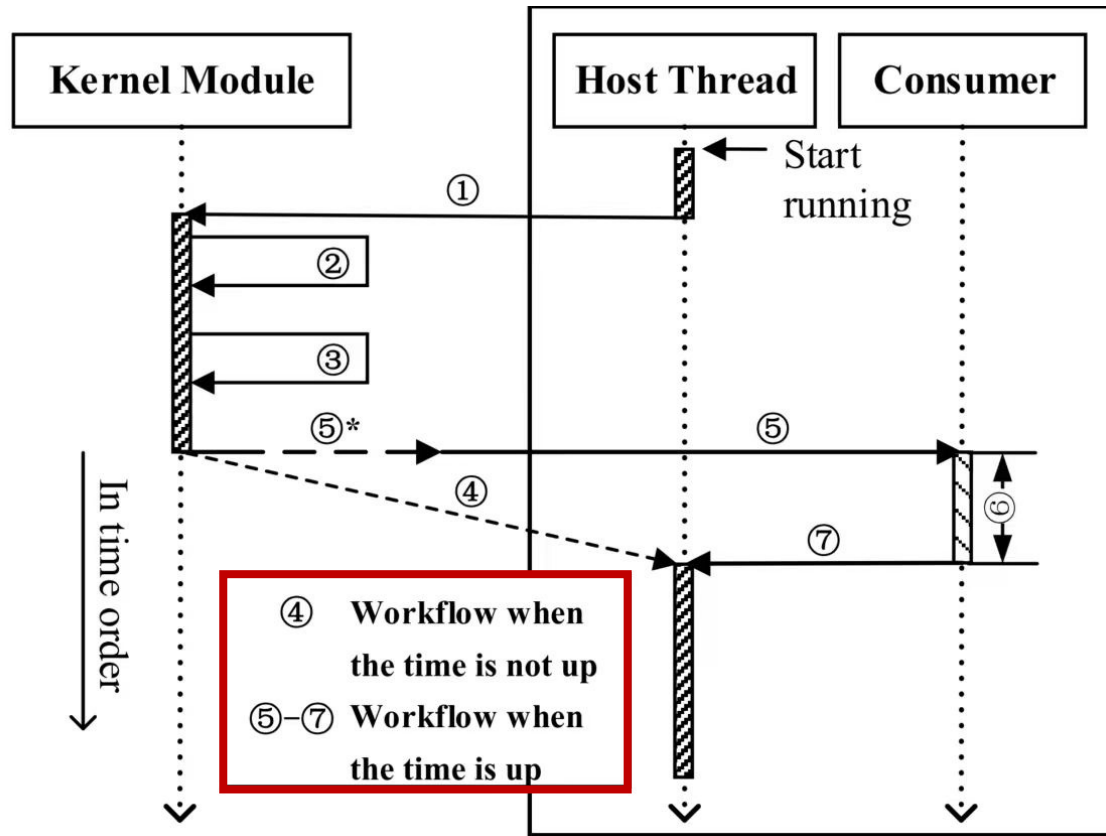
RT-NODROP Workflow



Periodic consumer invocation.

- 1) Bounded consumer WCET.
- 2) Bounded event residence time.
- 3) Bounded kernel buffer memory usage.

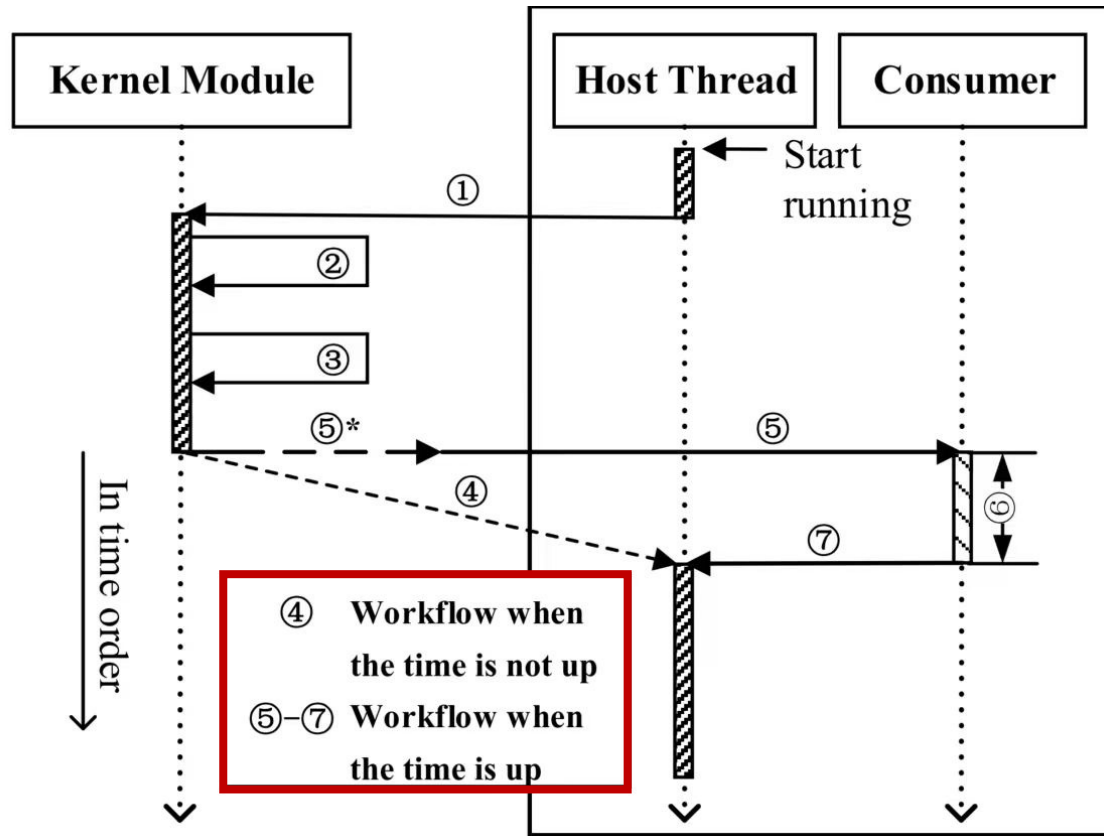
Real-Time Auditing Implementation



Task-aware event residence time control.

- 1) Individually controlled based on load and interference between applications.
- 2) Customize constraints and goals for event residence time.

Real-Time Auditing Implementation



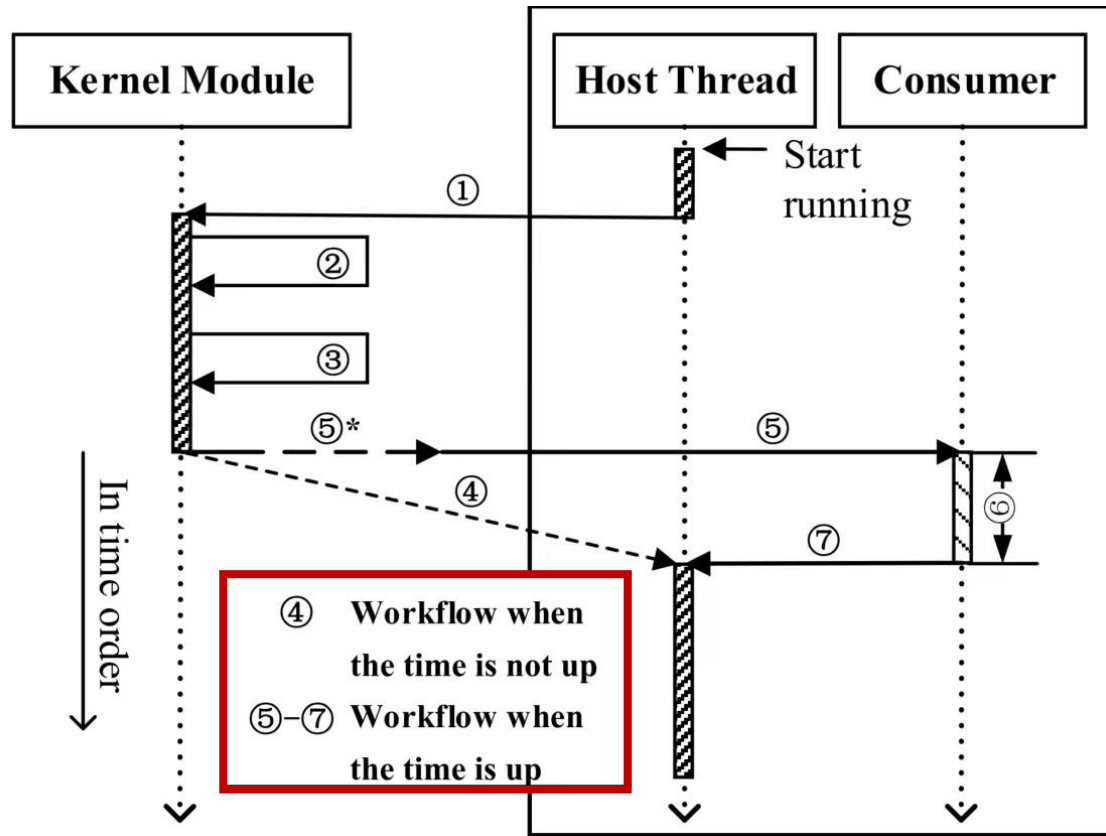
Workflow



Decoupling consumer invocation from syscall execution.

- 1) Uses a per-thread timer to asynchronously invoke a consumer.

Real-Time Auditing Implementation



IV

Consumer pre-loading.

- 1) Consumers are loaded in advance at the time of each thread creation.
- 2) Only has the cost of switching to the consumer, without the cost of loading and initialization.

Workflow

Response Time and Residence Time Analysis

- **Task Response Time (r_i) Analysis**

- (1) Increased task WCET. Δ is the maximum overhead imposed on each syscall, s_i is the number of syscalls per task. E_i is the WCET of task τ_i

$$E_i = e_i + s_i \times \Delta$$

- (2) Consumer execution interference. α is constant consumer invocation overhead, β is a processing overhead for each syscall. q_i is the consumer invocation period and ϵ_i is the consumer WCET

$$\epsilon_i = \alpha + \left\lceil \frac{q_i}{p_i} \right\rceil \times s_i \times \beta$$

- (3) The set of consumers with higher priority than τ_i is chp_i , the total consumer interference on τ_i is

$$I(\sigma_i) = \sum_{\sigma_k \in chp_i} \left\lceil \frac{r_i}{q_k} \right\rceil \times \epsilon_i$$

- (4) The complete response time under RT-NODROP is

$$r_i = E_i + b_i + \sum_{\tau_k \in hp_i} \left\lceil \frac{r_i}{p_k} \right\rceil \times E_k + I(\sigma_i)$$

We also analyze the response time of the existing auditing framework.

Response Time and Residence Time Analysis

- **Event Residence Time (R_i) Analysis**

- (1) Two components: the waiting time T_i^w in the kernel buffer and the processing time T_i^p inside the consumer

$$R_i = T_i^w + T_i^p$$

- (2) The worst-case waiting time is bounded by the consumer invocation period

$$T_i^w = q_i$$

- (3) Similar to the calculation of task response time, determining the consumer response time needs to combine the interference from both higher priority tasks and consumers.

$$T_i^p = \epsilon_i + \sum_{\tau_k \in hp_i} \left\lceil \frac{T_i^p}{p_k} \right\rceil \times E_k + \sum_{\sigma_k \in chp_i \setminus \sigma_i} \left\lceil \frac{T_i^p}{q_k} \right\rceil \times \epsilon_k$$

Response Time and Residence Time Analysis

Consumer Period Selection

Use PSO algorithm to adjust the consumer invocation period of each task.
Optimize the event residence time while ensuring the system schedulability.

```
1 def PSO_OPTIMIZE_QK(ts, qmin, qmax, N, max_gen, w,
2   c1, c2):
3     # Initialize particle swarm
4     ps = rand_int_vectors(|ts|, qmin, qmax, pop_size)
5     p_best = ps
6     g_best = argminp∈ps event_resident_time(p)
7     v = zeros(shape=ps.shape)
8     for iter in range(max_gen):
9         for i in range(N):
10            p = ps[i]
11            if is_schedulable(p, ts):
12                ert = event_residence_time(p, ts)
13            else:
14                ert = float('inf')
15            # Update personal best
16            if ert < event_residence_time(p_best[i], ts):
17                p_best[i] = p
18            # Update global best
19            if ert < event_residence_time(g_best, ts):
20                g_best = p
21            r1, r2 = random(), random()
22            v[i] = w · v[i] + c1 · r1 · (p_best[i] - p) +
23                c2 · r2 · (g_best - p)
24            ps[i] = clip(round(p + v[i]), qmin, qmax)
25    return g_best
```

Evaluation 1: Micro Benchmark

Syscall performance: The impact of RT-NODROP on syscalls is smaller than that of Sysdig and OMNILOG. The performance of RT-NODROP is up to 4 times higher than that of Ellipsis.

	stat	read	write	Open/close	null
Sysdig	1.85	1.10	1.00	3.69	0.86
Sysdig-dropfree	2.86	2.45	2.28	6.01	2.00
OMNILOG	3.80	3.28	3.19	7.80	3.06
Auditd	9.21	5.65	5.32	18.55	4.75
Ellipsis	7.37	4.52	4.26	14.84	3.80
NODROP	1.45	0.95	0.89	2.88	0.68
RT-NODROP	1.61	1.07	1.00	3.16	0.76

TABLE: lmbench results in microsecond. *Smaller is better.*

Evaluation 2: Schedulability and Event Residence Time

- Schedulability: RT-NODROP is superior over SOTA frameworks.**

- Event residence time: Remains stable, consistently around 10ms.**

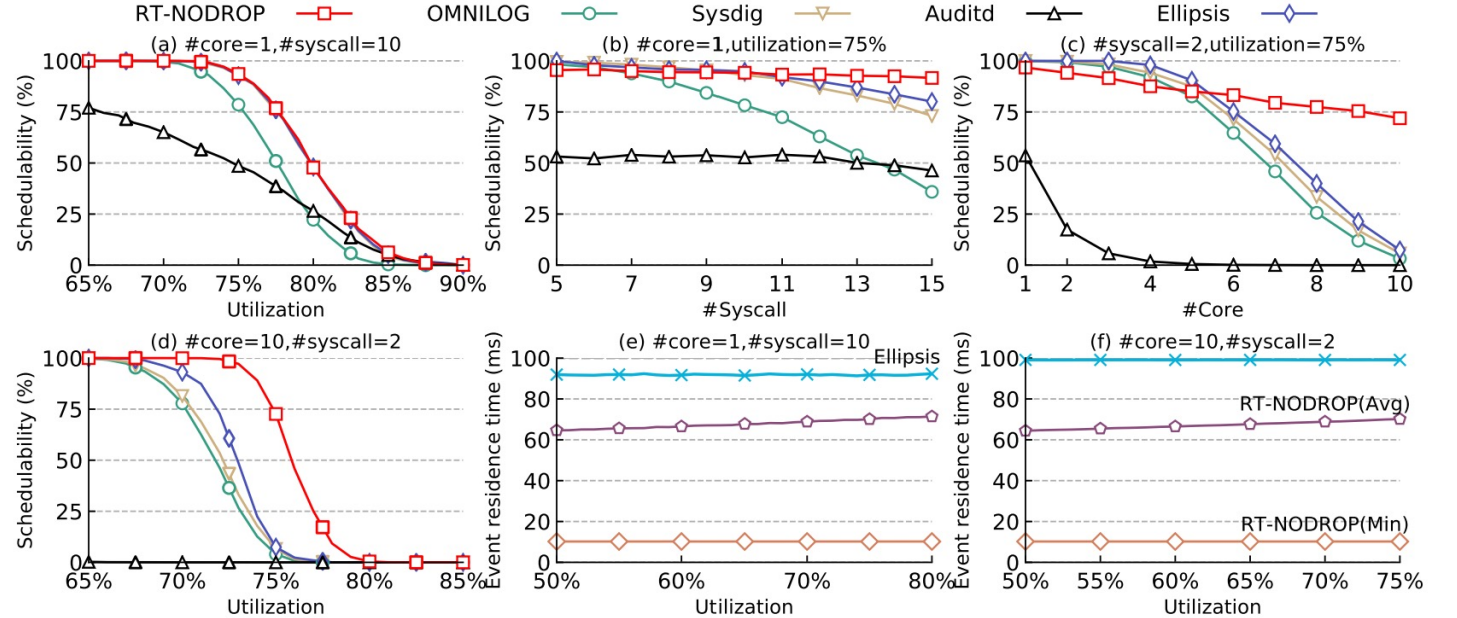


Figure: Response time and event residence time analysis.

On average, **RT-NODROP** outperforms Sysdig (OMNILOG) by **11.63% (21.98%)** and outperforms Ellipsis (Auditd) by **8.64% (850.97%)**.

	fig. 4(a)	fig. 4(b)	fig. 4(c)	fig. 4(d)
Sysdig	0.67%	3.24%	24.15%	18.45%
OMNILOG	8.85%	26.24%	32.38%	20.54%
Auditd	32.30%	80.00%	1342.85%	1948.73%
Ellipsis	0.35%	1.71%	18.82%	13.67%

TABLE: SUA improvement

Evaluation 3: Real-world Application (*Redis*)

RT-NODROP achieves up to **75.1% (138.86%, 324.6%)** higher throughput and **2.19× (3.07×, 5.02x)** lower 99.9th percentile tail latency than Sysdig (OMNILOG, Ellipsis).

RT-NODROP maintains a minimum event residence time around **10ms** without event dropping.

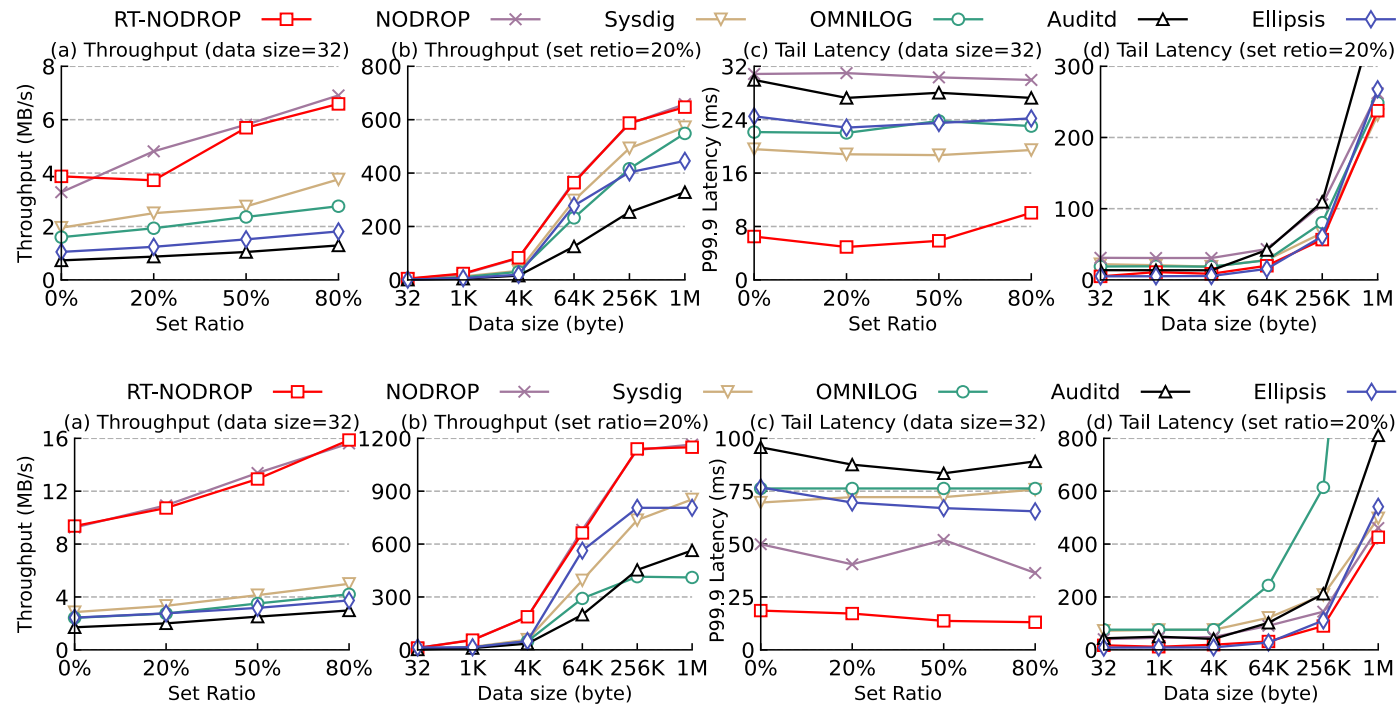


Figure: Redis performance on single core under different auditing frameworks

Evaluation 3: Real-world Application (*Redis*)

NODROP & RT-NODROP shows a similar distribution on both single-core and multi-core. Sysdig and OMNILOG have **worse** distribution and longer tail latency, Auditd and Ellipsis are **even worse**.

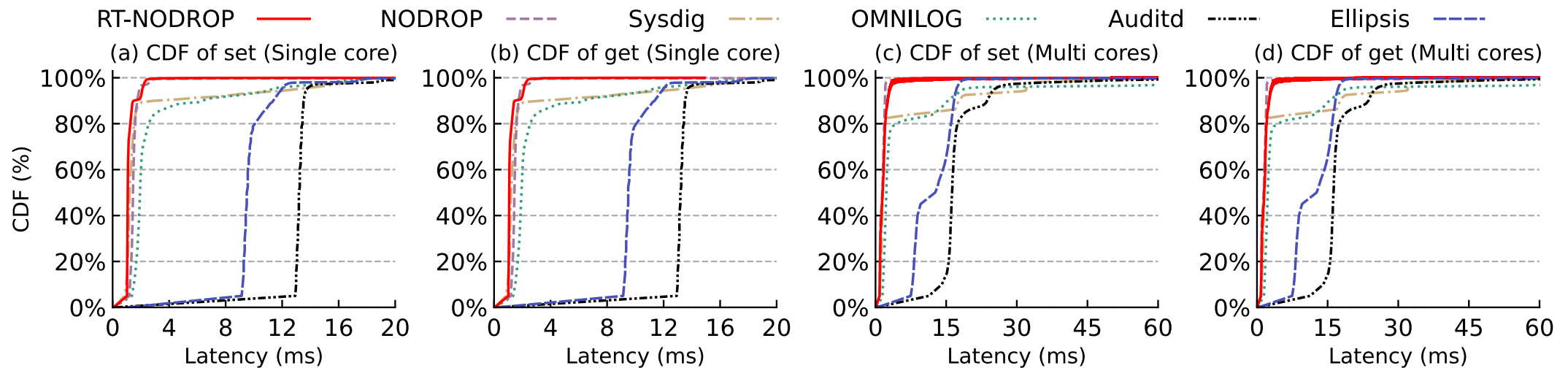
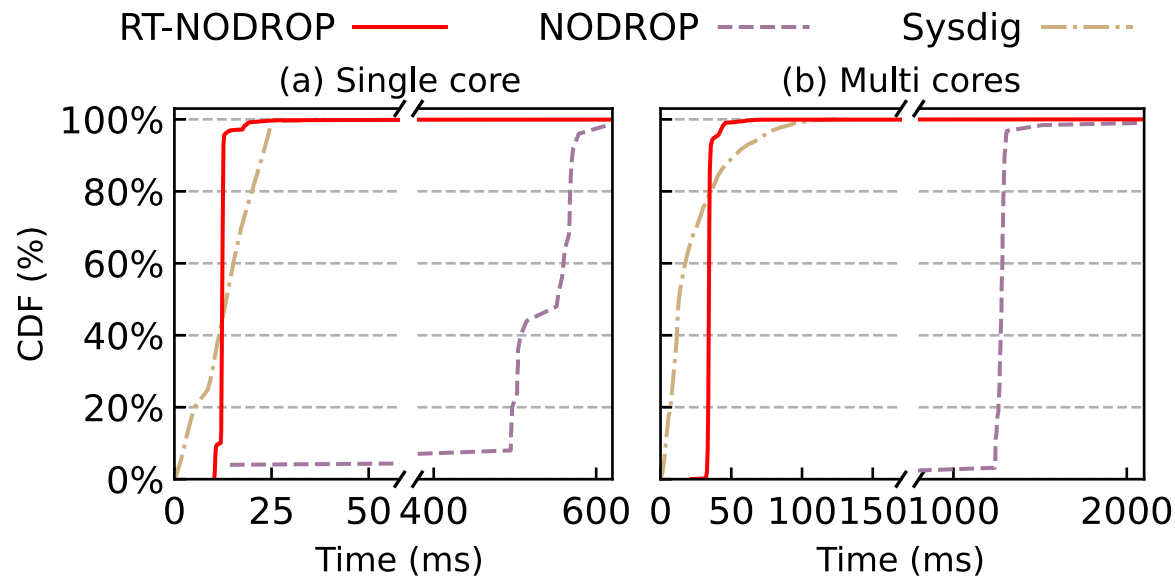


Figure: The cumulative distribution function of Redis set/get latency under different auditing frameworks.

Evaluation 3: Real-world Application (*Redis*)

When the Redis server is running on a single core, 99% of the RT-NODROP's events reside for less than **20ms** in the kernel buffer, which is much smaller than that of Sysdig and NODROP.



Conclusion

- Existing auditing frameworks fail to meet the predictability and security requirements of real-time systems.
 - They lack bounded worst-case execution time (WCET) and cannot integrate with real-time schedulability analysis, leading to unpredictable interference.
 - Attackers can exploit race conditions or flood audit buffers to tamper with or drop critical audit events, undermining system security.
- We propose **RT-NODROP**, which employs a periodic, threadlet-based design and integrates with Response Time Analysis to simultaneously achieve security and real-time guarantees.
- RT-NODROP improves schedulability by up to 117.9% and reduces Redis tail latency by up to $5.02\times$ compared to state-of-the-art frameworks (Sysdig, OMNILOG, Ellipsis).